

North South University

Boolean Algebra & Logic Gates

Digital Logic Design | CSC 231

Summer 2026 • Week 2.1

Mansur Mulk, PhD.



1. Boolean Algebra

- Postulates, axioms, duality principle

2. Boolean Functions & Truth Tables

- Representations, evaluation, complement

3. Logic Gates & Symbols

- AND, OR, NOT, NAND, NOR, XOR, XNOR

4. Algebraic Manipulation

- Theorems, simplification, consensus

5. Minterms & Maxterms

- Definitions, 3-variable tables, indices

6. Canonical Forms (SOM/POM)

- Sum of Minterms, Product of Maxterms

7. Standard SOP & POS

- Two-level implementations

8. Circuit Analysis & Synthesis

- Combinational circuits, NAND/NOR networks



Section 1

Boolean Algebra



What is Boolean Algebra?

Definition

- A closed algebraic system with set $K = \{0,1\}$ and two operators: $\cdot$ (AND) and $+$ (OR)
- Developed by George Boole (1854); applied to logic circuits by Claude Shannon (1938)

Why it matters for CSC 231

- Every digital circuit — from a simple gate to a CPU — is described by Boolean algebra
- Allows us to analyze, simplify, and implement switching functions as hardware

Two logic values only

- Logic 0 $\rightarrow$ low voltage ($\sim 0V$); Logic 1 $\rightarrow$ high voltage ($\sim 3.3V$ or $5V$)
- Binary signals simplify circuit implementation enormously

Variables & Literals

- Boolean variable: holds value 0 or 1 (e.g. A , B , x , y)
- Literal: a variable in true (A) or complemented (A') form



Postulates of Boolean Algebra

Postulate 1 — Closure

- If $A, B \in \{0,1\}$ then $A+B \in \{0,1\}$ and $A \cdot B \in \{0,1\}$

Postulate 2 — Identity Elements

- $A + 0 = A$ (identity for OR)
- $A \cdot 1 = A$ (identity for AND)

Postulate 3 — Commutativity

- $A + B = B + A$ $A \cdot B = B \cdot A$

Postulate 4 — Associativity

- $A+(B+C) = (A+B)+C$ $A \cdot (B \cdot C) = (A \cdot B) \cdot C$

Postulate 5 — Distributivity

- $A \cdot (B+C) = AB+AC$ $A+(B \cdot C) = (A+B) \cdot (A+C)$

Postulate 6 — Complement

- $A + A' = 1$ $A \cdot A' = 0$

Fundamental Theorems (Single Variable)



Theorem 1 — Idempotency

- $A + A = A$ $A \cdot A = A$

Theorem 2 — Null Element

- $A + 1 = 1$ $A \cdot 0 = 0$

Theorem 3 — Involution (Double Complement)

- $(A')' = A$

Properties of 0 and 1

- $0 + A = A$ $1 \cdot A = A$
- $1 + A = 1$ $0 \cdot A = 0$
- $0' = 1$ $1' = 0$

Complement

- $A + A' = 1$ $A \cdot A' = 0$



Theorems — Multi-Variable (Part 1)

Theorem 4 — Absorption

- $A + AB = A$ $A(A+B) = A$
 - Example: $(X+Y) + (X+Y)Z = X+Y$
 - Example: $AB'(AB'+B'C) = AB'$

Theorem 5 — Simplification

- $A + A'B = A + B$ $A(A'+B) = AB$
 - Example: $B + AB'C'D = B + AC'D$

Theorem 6 — Adjacency

- $AB + AB' = A$ $(A+B)(A+B') = A$
 - Example: $ABC + AB'C = AC$

Theorem 7

- $AB + AB'C = AB + AC$ $(A+B)(A+B'+C) = (A+B)(A+C)$
 - Example: $wy' + wx'y + wxy + wxz' = w + wxz' = w$



De Morgan's Theorem

First Theorem

- $(A + B)' = A' \cdot B'$ — NOT of a sum = product of NOTs

Second Theorem

- $(A \cdot B)' = A' + B'$ — NOT of a product = sum of NOTs

Generalized De Morgan

- $(A+B+\dots+Z)' = A' \cdot B' \cdot \dots \cdot Z'$
- $(A \cdot B \cdot \dots \cdot Z)' = A' + B' + \dots + Z'$

Extended Example

- $(a + bc)' = a'(bc)' = a'(b'+c') = a'b' + a'c'$
- $(a(b+c)+a'b)' = (ab+ac+a'b)' = (b+ac)' = b'(a'+c')$

Verification via Truth Table

- $x \quad y \quad (x+y)' \quad x'y' \quad (xy)' \quad x'+y'$
 - Columns $(x+y)'$ and $x'y'$ are identical — theorem confirmed



Theorem 8 — Consensus Theorem

Statement

- $AB + A'C + BC = AB + A'C$ (BC is the consensus term — redundant)
- $(A+B)(A'+C)(B+C) = (A+B)(A'+C)$

Proof (SOP form)

- $BC = BC \cdot 1 = BC(A+A') = ABC + A'BC$
- $AB + A'C + ABC + A'BC$
 - $= AB(1+C) + A'C(1+B) = AB + A'C \checkmark$

Examples from Chapter 2

- $AB + A'CD + BCD = AB + A'CD$
- $(a+b')(a'+c)(b'+c) = (a+b')(a'+c)$
- $ABC + A'D + B'D + CD = ABC + A'D + B'D$

Application

- Identifies and removes redundant terms in SOP/POS expressions
- Essential tool for algebraic minimization



Duality Principle & Summary Table

Duality Principle

- Swap AND $\leftrightarrow$ OR and 0 $\leftrightarrow$ 1 in any identity $\rightarrow$ another valid identity

Examples

- $A+0=A \leftrightarrow A \cdot 1=A$
- $A+1=1 \leftrightarrow A \cdot 0=0$
- $A+A'=1 \leftrightarrow A \cdot A'=0$
- $(A+B)'=A'B' \leftrightarrow (AB)'=A'+B'$

Why useful?

- Every Boolean law comes in a dual pair — learn one, get the other free

Quick Reference — Boolean Identities

- Identity: $A+0=A$, $A \cdot 1=A$
- Null: $A+1=1$, $A \cdot 0=0$
- Idempotent: $A+A=A$, $A \cdot A=A$
- Complement: $A+A'=1$, $A \cdot A'=0$
- Involution: $(A')'=A$
- Commutative: $A+B=B+A$
- Associative: $(A+B)+C=A+(B+C)$
- Distributive: $A(B+C)=AB+AC$
- Absorption: $A+AB=A$
- De Morgan: $(A+B)'=A'B'$
- Consensus: $AB+A'C+BC=AB+A'C$



Section 2

Boolean Functions & Truth Tables



Boolean Functions — Definition & Forms

Definition

- A Boolean function maps n binary input variables to a binary output: $F:\{0,1\}^n \rightarrow \{0,1\}$
- For n variables: 2^n possible input combinations; 2^{2^n} distinct functions

SOP — Sum of Products

- OR of product (AND) terms
 - Example: $f(A,B,C) = ABC + A'C + B'C$

POS — Product of Sums

- AND of sum (OR) terms
 - Example: $f(A,B,C) = (A'+B'+C')(A+C')(B+C')$

Representations

- 1. Algebraic expression 2. Truth table
- 3. Logic diagram (circuit) 4. Canonical (minterm/maxterm) form



Truth Table Construction — Example

x	y	z	y'	xy'	x'	x'z	f = xy' + x'z
0	0	0	1	0	1	0	0
0	0	1	1	0	1	1	1
0	1	0	0	0	1	0	0
0	1	1	0	0	1	1	1
1	0	0	1	1	0	0	1
1	0	1	1	1	0	0	1
1	1	0	0	0	0	0	0
1	1	1	0	0	0	0	0



Truth Table for $f(A,B,C) = AB + A'C + AC'$

A	B	C	AB	A'C	AC'	$f = AB+A'C+AC'$
0	0	0	0	0	0	0
0	0	1	0	1	0	1
0	1	0	0	0	0	0
0	1	1	0	1	0	1
1	0	0	0	0	1	1
1	0	1	0	0	0	0
1	1	0	0	0	1	1
1	1	1	1	0	0	1



Complementing Boolean Functions

Method 1 — Apply De Morgan's Theorem

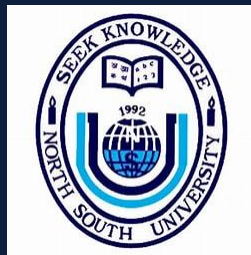
- Complement all literals AND swap $+$ $\leftrightarrow$ $\cdot$
 - Example: $F = xy' + x'z$
 - $F' = (xy')' \cdot (x'z)' = (x+y)(x+z')$

Method 2 — Dual + Complement Literals

- Step 1: Write the dual of F (swap $+$ $\leftrightarrow$ $\cdot$ and $0 \leftrightarrow 1$ in constants)
- Step 2: Complement every literal
 - Example: $F = AB + A'C$
 - Dual: $(A+B)(A'+C)$
 - Complement literals: $(A'+B')(A+C') = F'$ ✓

Switching Function Complement (from Ch.2)

- For $f(A,B,Q,Z) = \Sigma m(0,1,6,7)$
 - $f'(A,B,Q,Z) = \Sigma m(2,3,4,5,8,9,10,11,12,13,14,15)$



Section 3

Logic Gates & Symbols



AND, OR, and NOT Gates

AND Gate

- Output = 1 only when ALL inputs are 1
- Expression: $F = A \cdot B$ (or simply AB)
- Truth table: $00 \rightarrow 0$, $01 \rightarrow 0$, $10 \rightarrow 0$, $11 \rightarrow 1$

OR Gate

- Output = 1 when AT LEAST ONE input is 1
- Expression: $F = A + B$
- Truth table: $00 \rightarrow 0$, $01 \rightarrow 1$, $10 \rightarrow 1$, $11 \rightarrow 1$

NOT Gate (Inverter)

- Output = complement of input
- Expression: $F = A'$ Truth table: $0 \rightarrow 1$, $1 \rightarrow 0$

Physical Realisation

- Early computers used relay switches; today CMOS transistors implement all gates
- CMOS: Complementary Metal-Oxide Semiconductor — fast, low power

All Seven Gate Truth Tables



A	B	AND AB	OR A+B	NAND (AB)'	NOR (A+B)'	XOR $A \oplus B$	XNOR $(A \oplus B)'$
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1



NAND and NOR — Universal Gates

NAND as Universal Gate

- NOT A: $(A \cdot A)' = A'$
- AND AB: $((AB)')' = AB$
- OR A+B: $(A' \cdot B')' = A+B$ [De Morgan]

NOR as Universal Gate

- NOT A: $(A+A)' = A'$
- OR A+B: $((A+B)')' = A+B$
- AND AB: $(A'+B')' = AB$ [De Morgan]

Why Universal Gates Matter

- Any logic function can be built using ONLY NAND (or ONLY NOR) gates
- Reduces IC fabrication cost — only one gate type needed on chip

Active-High vs Active-Low Signals

- Active-high: signal asserted when HIGH (positive logic, L=0 H=1)
- Active-low: signal asserted when LOW (negative logic, L=1 H=0)



XOR and XNOR Gates

XOR — Exclusive OR

- $F = A \oplus B = A'B + AB'$ (output 1 when inputs DIFFER)
- POS form: $(A+B)(A'+B')$

XNOR — Exclusive NOR (Equivalence)

- $F = A \odot B = AB + A'B'$ (output 1 when inputs are SAME)
- $F = (A \oplus B)'$

Useful XOR Properties (from Ch.2 Eq.2.25–2.31)

- $A \oplus A = 0$ $A \oplus A' = 1$
- $A \oplus 0 = A$ $A \oplus 1 = A'$
- $A \oplus B = B \oplus A$ (commutative)
- $A \oplus (B \oplus C) = (A \oplus B) \oplus C$ (associative)

Application

- XOR: adders, parity generators/checkers, comparators
- XNOR: equality detectors, bit comparators



Logic Diagrams from Boolean Expressions

Two-Level AND-OR (SOP) Implementation

- Step 1 — Identify all literals and complements needed
- Step 2 — One AND gate per product term
- Step 3 — One OR gate combines all product terms

Example: $F = xy' + x'z$

- Gate 1 (NOT): generate y' Gate 2 (NOT): generate x'
- Gate 3 (AND): $x \cdot y' \rightarrow xy'$
- Gate 4 (AND): $x' \cdot z \rightarrow x'z$
- Gate 5 (OR): $xy' + x'z \rightarrow F$

Two-Level OR-AND (POS) Implementation

- One OR gate per sum term, then AND all together

Gate Delay

- SOP / POS = exactly 2 gate levels (AND then OR, or OR then AND)
- Total delay = $2 \times$ per-gate propagation delay



Section 4

Algebraic Manipulation & Simplification



Expression Simplification — Strategy

Goal

- Reduce number of literals and terms $\rightarrow$ fewer gates, lower cost, faster circuit

Key Techniques

- 1. Combining (Adjacency): $AB + AB' = A$
- 2. Absorption: $A + AB = A$
- 3. Simplification: $A + A'B = A + B$
- 4. De Morgan: convert between sum/product forms
- 5. Consensus: remove redundant terms

Operator Precedence

- Highest: NOT (complement) $\rightarrow$ then AND $\rightarrow$ then OR
- $A+B \cdot C$ means $A+(B \cdot C)$ — AND binds tighter than OR

General Approach

- Factor common literals, group adjacently differing terms, then apply absorption



Simplification — Worked Examples (Part 1)

Example 1: $F = AB + AB'C + A'C$

- $AB + AB'C = A(B + B'C) = A(B + C)$ [T5(a)]
- $F = A(B+C) + A'C = AB + AC + A'C$
 - $= AB + C(A+A') = AB + C$ ✓

Example 2 (from Ch.2 T7): $wy' + wx'y + wxyz + wxz'$

- $= wy' + wx'y + wxy + wxz'$ [T7(a): add missing y]
- $= wy' + wy + wxz'$ [T7(a): $x'y + xy = y$]
- $= w + wxz'$ [T7(a): $y' + y = 1$]
- $= w$ [Absorption T4(a)] ✓

Example 3: $F = ABC + AB'C + A'B'C + A'BC$

- $ABC + AB'C = AC$ $A'B'C + A'BC = A'C$
- $F = AC + A'C = C$ ✓



Simplification — Worked Examples (Part 2)

Example 4 (De Morgan application)

- $(a + bc)' = a'(bc)' = a'(b' + c') = a'b' + a'c'$
- Note: $(a+bc)' \neq a'b'+c'$ (common mistake!)

Example 5 (Nested De Morgan)

- $(a(b+z(x+a'))))' = a' + (b+z(x+a'))'$
 - $= a' + b'(z(x+a'))'$
 - $= a' + b'(z' + (x+a'))'$
 - $= a' + b'(z' + x'a)$
 - $= a' + b'(z' + x')$ [T5(a): $x'a \rightarrow x'$ since $a'+x'a=a'+x'$]

Example 6 (Consensus removal)

- $AB + A'CD + BCD = AB + A'CD$ [T9(a): BCD is consensus]

Example 7: $F = (A+B)(A'+C)(B+C)$

- $(A+B)(A'+C)(B+C) = (A+B)(A'+C)$ [consensus T9(b)]



Shannon's Expansion Theorem

Theorem 10 — Shannon's Expansion

- $f(x_1, x_2, \dots, x_n) = x_1 \cdot f(1, x_2, \dots, x_n) + x_1' \cdot f(0, x_2, \dots, x_n)$ [cofactor expansion]
- $f(x_1, x_2, \dots, x_n) = [x_1 + f(0, x_2, \dots, x_n)] \cdot [x_1' + f(1, x_2, \dots, x_n)]$

Example: Expand $f(A, B, C) = AB + AC' + A'C$

- $f = A \cdot f(1, B, C) + A' \cdot f(0, B, C)$
 - $= A(B + C') + A'(C)$
- Expand again on B:
 - $= B[A + A'C] + B'[AC' + A'C]$
 - $= AB + A'BC + AB'C' + A'B'C$

Alternative: Add Missing Literals (Theorem 6)

- $AB = AB(C + C') = ABC + ABC' = m_7 + m_6$
- $AC' = AC'(B + B') = ABC' + AB'C' = m_6 + m_4$
- $A'C = A'C(B + B') = A'BC + A'B'C = m_3 + m_1$
- Therefore $f(A, B, C) = \Sigma m(1, 3, 4, 6, 7)$



Section 5

Minterms & Maxterms



Minterms — Definition & Indexing

Definition

- A minterm is a product (AND) term where each of the n variables appears exactly ONCE (complemented or uncomplemented)
- For n variables: exactly 2^n minterms, indexed 0 through 2^n-1

Index Rule for Minterms

- '1' in the binary index $\rightarrow$ variable appears uncomplemented
- '0' in the binary index $\rightarrow$ variable appears complemented

Two-Variable Minterms (A, B)

- $m_0 = A'B'$ (00) $m_1 = A'B$ (01)
- $m_2 = AB'$ (10) $m_3 = AB$ (11)

Key Property

- Each minterm equals 1 for exactly ONE input combination
- The sum of ALL minterms = 1 for any set of variable values



Three-Variable Minterms & Maxterms Table

Index	x	y	z	Minterm m_i	Maxterm M_i	Relation
0	0	0	0	$x'y'z'$	$x+y+z$	$m_i = M_i'$
1	0	0	1	$x'y'z$	$x+y+z'$	$m_i = M_i'$
2	0	1	0	$x'yz'$	$x+y'+z$	$m_i = M_i'$
3	0	1	1	$x'yz$	$x+y'+z'$	$m_i = M_i'$
4	1	0	0	$xy'z'$	$x'+y+z$	$m_i = M_i'$
5	1	0	1	$xy'z$	$x'+y+z'$	$m_i = M_i'$
6	1	1	0	xyz'	$x'+y'+z$	$m_i = M_i'$
7	1	1	1	xyz	$x'+y'+z'$	$m_i = M_i'$



Maxterms — Definition & Indexing

Definition

- A maxterm is a sum (OR) term where each of the n variables appears exactly ONCE (complemented or uncomplemented)
- Dual of minterm: uses $+$ instead of $\cdot$

Index Rule for Maxterms (OPPOSITE of minterms)

- '0' in the binary index $\rightarrow$ variable appears uncomplemented
- '1' in the binary index $\rightarrow$ variable appears complemented

Two-Variable Maxterms (A, B)

- $M_0 = A+B$ (00) $M_1 = A+B'$ (01)
- $M_2 = A'+B$ (10) $M_3 = A'+B'$ (11)

Key Property

- Each maxterm equals 0 for exactly ONE input combination
- The product of ALL maxterms = 0

Fundamental Relationship

- $(m_i)' = M_i$ and $(M_i)' = m_i$ for the same index i



Purpose of the Index

Index encodes the input combination

- Treat the variable combination as a binary number
- The index tells you the single row where that minterm/maxterm is special

Minterm m_i

- Equals 1 ONLY when inputs match the binary representation of i
- All other $2^n - 1$ combinations give $m_i = 0$

Maxterm M_i

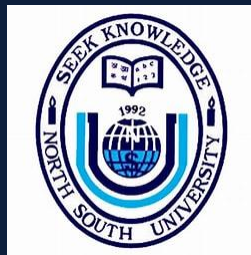
- Equals 0 ONLY when inputs match the binary representation of i
- All other $2^n - 1$ combinations give $M_i = 1$

Example — Three Variables

- $m_5 = xy'z$: equals 1 only when $x=1, y=0, z=1$ (binary 101 = 5) ✓
- $M_5 = x' + y + z'$: equals 0 only when $x=1, y=0, z=1$ ✓

Summary

- There are 2^n minterms AND 2^n maxterms for n -variable functions



Section 6

Canonical Forms — SOM & POM



Sum-of-Minterms (SOM) Canonical Form

Definition

- Express F as an OR of minterms for all rows where $F = 1$
- Also called: Canonical SOP or minterm expansion

Notation

- $F(x,y,z) = \Sigma m(2,3,5,7)$

Derivation from Truth Table ($f = xy' + x'z$ example)

- $F=1$ at rows: 1,3,4,5 $\rightarrow f = m_1 + m_3 + m_4 + m_5$
 - $= x'y'z + x'yz + xy'z' + xy'z$
 - $= \Sigma m(1,3,4,5)$

From Chapter 2 Example

- $f_1(A,B,C) = A'BC' + ABC' + A'BC + ABC = m_2 + m_3 + m_6 + m_7 = \Sigma m(2,3,6,7)$

Key Property

- Every Boolean function has a UNIQUE canonical SOM
- Focus only on the '1' entries of the truth table



Product-of-Maxterms (POM) Canonical Form

Definition

- Express F as an AND of maxterms for all rows where $F = 0$
- Also called: Canonical POS or maxterm expansion

Notation

- $F(x,y,z) = \prod M(0,2,6)$

Derivation from Truth Table (same example)

- $F=0$ at rows: 0,2,6 $\rightarrow f = M_0 \cdot M_2 \cdot M_6$
 - $= (x+y+z)(x+y'+z)(x'+y'+z)$
 - $= \prod M(0,2,6)$

From Chapter 2 Example

- $f_2(A,B,C) = (A+B+C)(A+B+C')(A'+B+C)(A'+B+C') = \prod M(0,1,4,5)$

Key Property

- Every Boolean function has a UNIQUE canonical POM
- Focus only on the '0' entries of the truth table

SOM / POM Truth Table Side-by-Side



x	y	z	f	Minterm (for F=1)	Maxterm (for F=0)
0	0	0	0	—	$M_0 = x+y+z$
0	0	1	0	—	$M_1 = x+y+z'$
0	1	0	1	$m_2 = x'yz'$	—
0	1	1	1	$m_3 = x'yz$	—
1	0	0	0	—	$M_4 = x'+y+z$
1	0	1	1	$m_5 = xy'z$	—
1	1	0	0	—	$M_6 = x'+y'+z$
1	1	1	1	$m_7 = xyz$	—



Conversions Between Canonical Forms

SOM $\rightarrow$ POM

- Minterm indices where $F=1 \leftrightarrow$ Maxterm indices where $F=0$
- The two sets are complementary — they partition $\{0,1,\dots,2^n-1\}$

Rule

- If $F = \Sigma m(i_1, i_2, \dots)$ then $F = \Pi M(\text{all remaining indices})$

Example (from Ch.2 Eq.2.10)

- $f_1(A,B,C) = \Sigma m(2,3,6,7) = \Pi M(0,1,4,5)$
- $f_2(A,B,C) = \Pi M(0,1,4,5) = \Sigma m(2,3,6,7)$ — same function!

Function Complement

- If $F = \Sigma m(S)$ then $F' = \Pi M(S)$ — same indices, different symbol
- Example: $F = \Sigma m(2,3,6,7) \rightarrow F' = \Pi M(2,3,6,7)$
- Equivalently: $F' = \Sigma m(0,1,4,5)$

Generalisation

- To convert: interchange $\Sigma \leftrightarrow \Pi$ and list the MISSING indices



Deriving Canonical Forms Algebraically

Method — Expand Using Theorem 6 ($ab+ab'=a$)

- Multiply each product term by $(X+X')$ for every missing variable X

Example: $f(A,B,C) = AB + AC' + A'C \rightarrow$ canonical SOP

- $AB = AB(C+C') = ABC + ABC' = m_7 + m_6$
- $AC' = AC'(B+B') = ABC' + AB'C' = m_6 + m_4$
- $A'C = A'C(B+B') = A'BC + A'B'C = m_3 + m_1$
- $f = \sum m(1,3,4,6,7)$ (remove duplicate m_6)

Example: $f(A,B,C) = A(A+C') \rightarrow$ canonical POS

- $A = (A+B)(A+B')$ then expand each: $(A+B'+C')(A+B'+C)(A+B+C')(A+B+C) = M_3M_2M_1M_0$
- $(A+C') = (A+B'+C')(A+B+C') = M_3M_1$
- $f = \prod M(0,1,2,3)$

Don't-Care Conditions

- Some input combinations may never occur $\rightarrow$ don't-care minterms d_i
- $f(A,B,C) = \sum m(0,3,7) + d(4,5) = M(1,2,6) \cdot D(4,5)$



Section 7

Standard SOP & POS Forms



Standard SOP and POS — Definitions

Standard SOP (Sum of Products)

- OR of AND terms; each term may have FEWER than n variables
- More general than canonical SOP — not every term needs to be a minterm
 - Example: $F = AB + C$ (standard SOP, 2 terms)
 - Canonical SOP of same function: $F = ABC + ABC' + AB'C + A'BC + \dots$ (more terms)

Standard POS (Product of Sums)

- AND of OR terms; each factor may have fewer than n variables
 - Example: $F = (A+C)(A'+B)$ (standard POS)

Canonical vs Standard

- Canonical: unique, each term has ALL n variables
- Standard: not unique, terms may have fewer — result of simplification
- Standard forms use fewer gates (the whole point of simplification!)

Converting Standard → Canonical

- Expand: for any term T missing variable X , write $T = TX + TX'$



Two-Level Gate Implementations

AND-OR Network (implements SOP)

- Level 1: AND gates (one per product term)
- Level 2: single OR gate
- Fast — exactly 2 gate delays from input to output

OR-AND Network (implements POS)

- Level 1: OR gates (one per sum term)
- Level 2: single AND gate

NAND-NAND Network (equivalent to AND-OR)

- Replace AND-OR by NAND-NAND using De Morgan: $F = ((AB)'(CD)')' = AB + CD$
- Preferred in practice — NAND gates are cheapest to fabricate

NOR-NOR Network (equivalent to OR-AND)

- Replace OR-AND by NOR-NOR using De Morgan

Implementation Procedure for NAND

- 1. Write function as SOM 2. Write minterms 3. Simplify to SOP
- 4. Transform to NAND form 5. Draw NAND diagram



NAND Network — Worked Example

Example (from Ch.2): $f(X,Y,Z) = \Sigma m(0,3,4,5,7)$

- Step 1 & 2 — write minterms:
 - $f = m_0 + m_3 + m_4 + m_5 + m_7 = X'Y'Z' + X'YZ + XY'Z' + XY'Z + XYZ$
- Step 3 — simplify using Theorem 6:
 - $XY'Z' + XY'Z = XY'$; $X'Y'Z' + XY'Z' = Y'Z'$
 - $X'YZ + XYZ = YZ$
 - $f = XY' + Y'Z' + YZ$
- Step 4a — NAND form (using Theorem 4):
 - $f = ((XY') \cdot (Y'Z')) \cdot (YZ)'$
- Step 4b — NAND form (using De Morgan / Theorem 8):
 - $f = (XY' + Y'Z' + YZ)'' = \text{NAND of NAND of each product term}$
- Step 5 — draw: three 2-input NAND gates feeding one 3-input NAND gate



Circuit Synthesis — Design Examples

Burglar Alarm (Ex.2.41)

- A = secret switch closed
- B = safe in normal position
- C = clock between 10:00–14:00
- D = closet door closed

Logic equation

- Alarm = $AB' + DC' + DA'$
- (safe moved AND secret switch ON)
- (OR closet opened after hours)
- (OR closet opened, switch open)

Smoke Alarm (Ex.2.38)

- S_1, S_2 = smoke detectors (active-low)
- Sprinkler: $S_1 \text{ NOR } S_2$
- Fire dept: $S_1 \text{ NAND } S_2$

Voter Circuit (Ex.2.42)

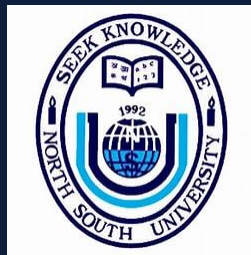
- Family vote: hamburger=0, chicken=1
- Majority wins; if Mom & Dad agree, they win
- A=Dad, B=Mom, C=Joe, D=Sue
- $F = AB + ACD + BCD$

2-bit Adder (Ex.2.43)

- Inputs: $A_1A_0 + B_1B_0$
- Outputs: $C_1S_1S_0$
- $S_0 = A_0 \oplus B_0$
- $S_1 = A_1 \oplus B_1 \oplus (A_0B_0)$
- $C_1 = A_1B_1 + A_1A_0B_0 + B_1A_0B_0$

Key insight

- Word description $\rightarrow$ equations $\rightarrow$ gates
- Always verify with truth table



Section 8

Analysis of Combinational Circuits

Purpose of Analysis

- Determine the behaviour and verify correctness of an existing circuit
- Convert circuit to a different implementation form

Method 1 — Algebraic Method

- Write a switching expression for each gate output (from inputs forward)
- Combine gate expressions to get the overall output expression
- Simplify using Boolean algebra theorems

Method 2 — Truth Table Method

- Evaluate the output one gate at a time for every input combination
- Fill in the truth table row by row

Method 3 — Timing Diagram Analysis

- Read input waveforms over time
- Propagate through gates level by level
- Verify output waveform matches expected function

Algebraic Analysis — Example (Ex.2.33)



Given circuit for $f(a,b,c)$

- Internal signal $g_1 = a \oplus b$ (XOR of a and b)
- Internal signal $g_2 = g_1 \cdot c$ (AND)
- Internal signal $g_3 = a \cdot b$ (AND)
- Output: $f = g_2 + g_3$

Simplify

- $f = (a \oplus b)c + ab$
 - $= (a'b + ab')c + ab$ [Eq.2.24]
 - $= a'bc + ab'c + ab$ [distribute]
 - $= a'bc + a(b'c + b)$ [factor a]
 - $= a'bc + a(b+c)$ [T5: $b'c + b = b + c$]
 - $= a'bc + ab + ac$
 - $= bc + ab + ac$ [T5: $a'bc + ac \rightarrow c(a' + a)b$? check]

Result

- $f(a,b,c) = ab + ac + bc$ (majority function!) ✓



Propagation Delay

Definition

- The delay between an input change and the corresponding output change
- Physical characteristic of every real logic gate

Two Key Parameters

- t_{PLH} = propagation delay, low-to-high output transition
- t_{PHL} = propagation delay, high-to-low output transition
- Approximation: $t_p \approx (t_{PLH} + t_{PHL}) / 2$

Multi-Level Delay

- Total delay $\approx$ (number of gate levels) $\times$ (per-gate delay)
- Two-level SOP/POS: delay = $2 \times t_p$ (fastest for complex functions)

Logic Families (from Ch.2 Table 2.7)

- TTL (74LS): $\sim 5\text{--}10$ ns per gate, ~ 2 mW per gate
- CMOS: $\sim 1\text{--}5$ ns, extremely low static power
- ECL: fastest (~ 1 ns), highest power consumption

Fan-in & Fan-out

- Fan-in: max inputs a gate can accept without degraded performance

Algebraic Forms — Complete Summary



Literal

- A variable complemented or uncomplemented: A , A' , x , x'

Product Term

- Literals ANDed: AB , $AB'C$, $x'y$

Sum Term

- Literals ORed: $A+B$, $A'+B'+C$, $x+y'$

Minterm

- Product term with ALL n variables: $AB'C$ (3 vars)

Maxterm

- Sum term with ALL n variables: $A+B'+C$ (3 vars)

Canonical SOP

- Sum (OR) of minterms only: $\Sigma m(\text{indices})$

Canonical POS

- Product (AND) of maxterms only: $\Pi M(\text{indices})$

Standard SOP

- OR of product terms (may lack some variables)

Standard POS

- AND of sum terms (may lack some variables)

Conversion Rule

- $\Sigma m(S) = \Pi M(\text{complement of } S)$

SOP $\rightarrow$ POS Algebraically

- Use $A+(BC) = (A+B)(A+C)$

POS $\rightarrow$ SOP Algebraically

- Expand $(A+B)(C+D) = AC+AD+BC+BD$

Don't-Care

- $f = \Sigma m(\text{list}) + d(\text{don't-care list})$
- $= \Pi M(\text{list}) \cdot D(\text{don't-care list})$

Complement Identity

- $(m_i)' = M_i$ $(M_i)' = m_i$



Key Takeaways — Boolean Algebra for CSC 231

Boolean Algebra

- 6 postulates define the system
- 10 theorems for simplification
- De Morgan & Consensus are most powerful
- Duality: every law has a dual

Logic Gates

- AND, OR, NOT are fundamental
- NAND and NOR are universal (build anything)
- XOR/XNOR for arithmetic and comparison
- Two-level = 2 gate delays (optimal speed)

Truth Tables

- Unique — one table per function
- Bridge between word description & algebra

Minterms & Maxterms

- $m_i = 1$ for exactly one row; $M_i = 0$ for exactly one row
- $(m_i)' = M_i$ — fundamental duality
- 2^n minterms and maxterms for n variables

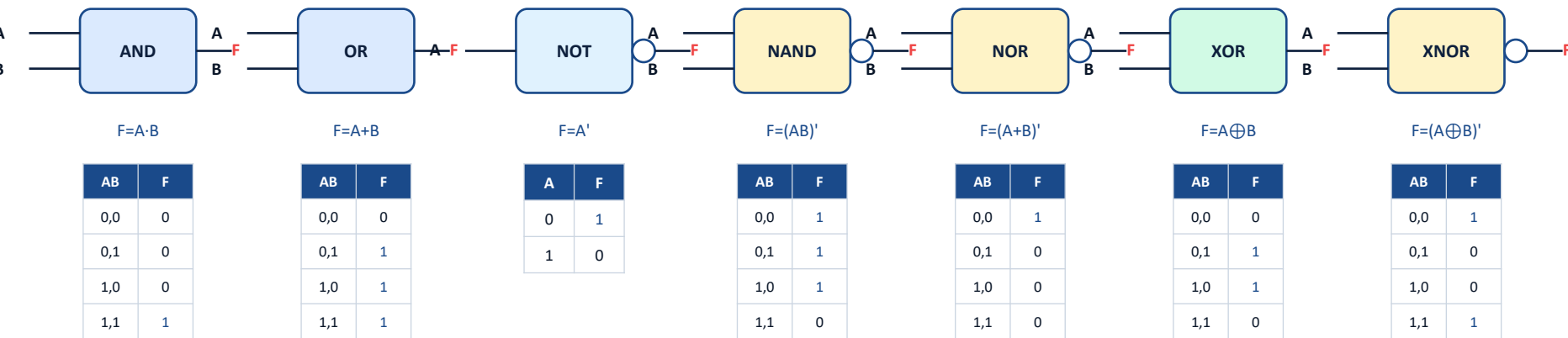
Canonical Forms

- SOM: $\sum m(\text{rows where } F=1)$ — unique
- POM: $\prod M(\text{rows where } F=0)$ — unique
- Complementary index sets

Standard Forms & Minimization

- Standard < Canonical in gate count
- Algebraic tools: adjacency, absorption, consensus
- Always verify result matches truth table

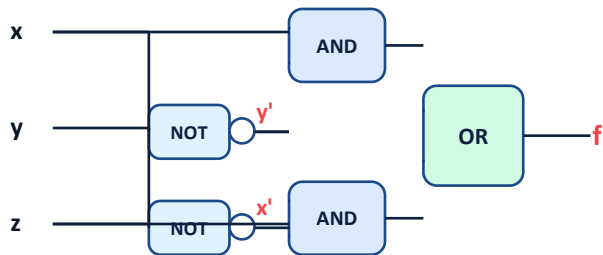
Slide 50 — Logic Gate Symbols & IEEE Standard Notation



Note: Circle (o) on output = inversion (NOT) | AND: flat back · OR: curved back · XOR: extra curved line

Slide 51 — Logic Diagrams: $f = xy' + x'z$ and $f = x(y' + z)$

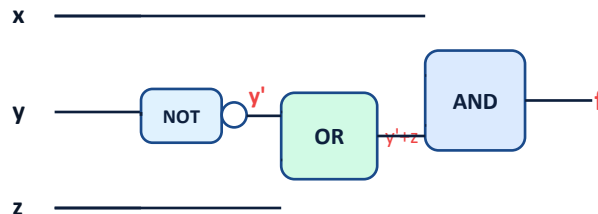
Circuit 1: $f = xy' + x'z$ (SOP — AND-OR, 2 levels)



$$f = xy' + x'z$$

- 2 NOT + 2 AND + 1 OR = 5 gates
- Two gate-delay levels (AND → OR)

Circuit 2: $f = x(y' + z)$ (factored — 3 levels)



$$f = x(y' + z) \text{ [factored SOP]}$$

- 1 NOT + 1 OR + 1 AND = 3 gates
- Three gate-delay levels; fewer gates

Key Insight: Both circuits implement the same function $f = xy' + x'z = x(y' + z)$. SOP (Circuit 1) has 2 gate delays but uses 5 gates. Factored form (Circuit 2) uses 3 gates but introduces 3 levels of delay.

Slide 52 — Combinational Circuits & Two-Level Implementations

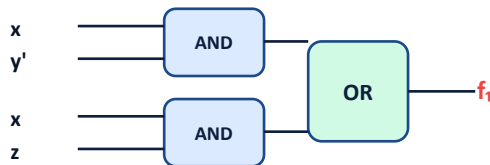
Combinational Circuit Model



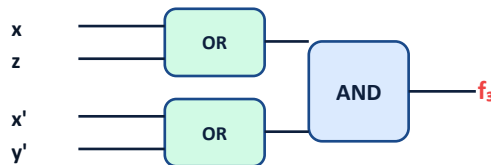
- n inputs, m outputs
- Each output = function of present inputs only
- No feedback / memory (combinational = memoryless)

Two-Level Gate Implementations

$$f_1 = xy' + xz \text{ (AND-OR / SOP)}$$



$$f_3 = (x+z)(x'+y') \text{ (OR-AND / POS)}$$



AND-OR (SOP): AND gates in Level 1, single OR in Level 2. Implements any SOP expression directly. **OR-AND (POS):** OR gates in Level 1, single AND in Level 2. Implements any POS expression directly.

NAND-NAND $\equiv$ AND-OR | NOR-NOR $\equiv$ OR-AND (both use De Morgan) | Always exactly 2 gate-delay levels

Slide 53 — NAND & NOR Universal Gates: Building AND, OR, NOT

NAND-based Constructions (using De Morgan: $(AB)' = A' + B'$)

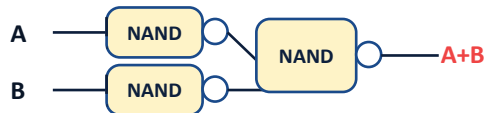
NOT from NAND: $A \text{ NAND } A = A'$



AND from NAND: $((AB)')' = AB$



OR from NAND: $A' \text{ NAND } B' = A + B$



NOR-based Constructions (using De Morgan: $(A+B)' = A'B'$)

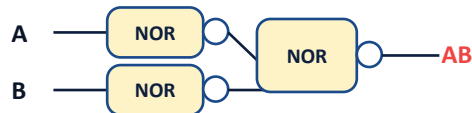
NOT from NOR: $A \text{ NOR } A = A'$



OR from NOR: $((A+B)')' = A+B$



AND from NOR: $A' \text{ NOR } B' = AB$



Why Universal Gates? Any Boolean function can be implemented using ONLY NAND gates, or ONLY NOR gates.

This reduces IC fabrication cost — manufacturers need to produce only ONE type of gate.

NAND: preferred for TTL/CMOS SOP circuits · NOR: preferred for CMOS POS circuits · NAND-NAND = AND-OR · NOR-NOR = OR-AND

Slide 54 — SOM & POM: Full Worked Example

Function: $f(x,y,z)$ from truth table

x	y	z	f	Minterm / Maxterm
0	0	0	0	$M_0 = x+y+z$
0	0	1	0	$M_1 = x+y+z'$
0	1	0	1	$m_2 = x'yz'$
0	1	1	1	$m_3 = x'yz$
1	0	0	0	$M_4 = x'+y+z$
1	0	1	1	$m_5 = xy'z$
1	1	0	0	$M_6 = x'+y'+z$
1	1	1	1	$m_7 = xyz$

F=1 $\rightarrow$ contributes to SOM
F=0 $\rightarrow$ contributes to POM

Canonical Forms

Sum-of-Minterms (SOM) = $\Sigma m(2,3,5,7)$

$$f = x'yz' + x'yz + xy'z + xyz$$

$$= \Sigma m(2,3,5,7)$$

Product-of-Maxterms (POM) = $\Pi M(0,1,4,6)$

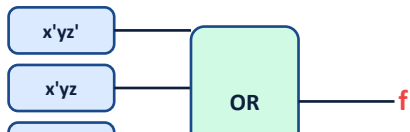
$$f = (x+y+z)(x+y+z')(x'+y+z)(x'+y'+z)$$

$$= \Pi M(0,1,4,6)$$

Conversion Identity: $\Sigma m(2,3,5,7) = \Pi M(0,1,4,6)$

Complement: $f' = \Sigma m(0,1,4,6) = \Pi M(2,3,5,7) \mid (m_i)' = M_i$

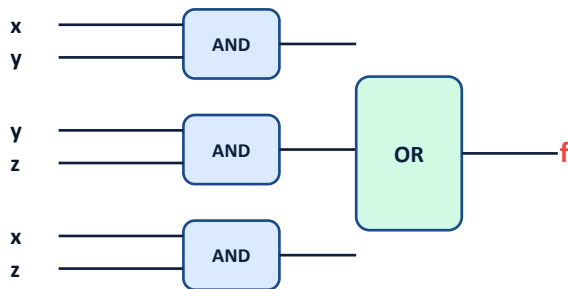
Circuit: $f = x'yz' + x'yz + xy'z + xyz$ (4-input AND-OR)



Steps: (1) Truth table $\rightarrow$ identify $F=1$ rows $\rightarrow$ (2) write minterm for each $\rightarrow$ (3) OR all minterms = canonical SOM

Slide 55 — Circuit Analysis & 2-Level vs 3-Level Implementation

Algebraic Circuit Analysis: Derive f from gates



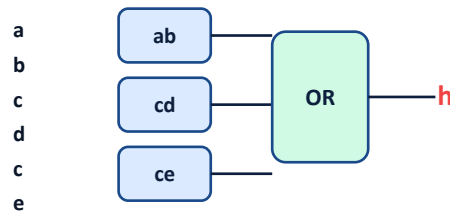
Step-by-step:

$g_1 = xy$
 $g_2 = yz$
 $g_3 = xz$
 $f = g_1 + g_2 + g_3$
 $f = xy + yz + xz$

Result: $f(x,y,z) = xy + yz + xz$ (Majority function — $F=1$ when ≥ 2 inputs are 1)
Canonical SOM: $\Sigma m(3,5,6,7)$ | Canonical POM: $\Pi M(0,1,2,4)$

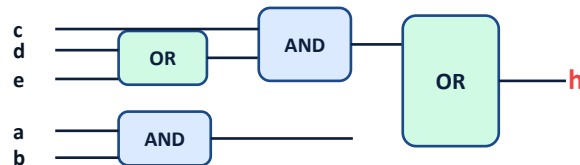
2-Level vs 3-Level: $h = ab + cd + ce$

2-Level (SOP): $h = ab + cd + ce$



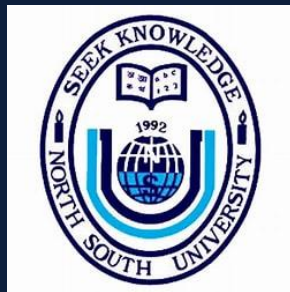
3 AND + 1 OR | 2 gate levels | 6 literals

3-Level (factored): $h = ab + c(d+e)$



1 OR + 2 AND + 1 OR | 3 levels | 5

Trade-off: 2-Level SOP is faster (2 gate delays) but uses more gates (more chip area). Factored 3-Level uses fewer gates (lower cost) but is slower (3 delays). *Fan-in constraints (max inputs per gate) also force multi-level designs on large circuits.*



Thank You

Questions & Discussion — CSC 231 Boolean Algebra